deadenv — Product Requirements Document

> Replace `.env` files with encrypted, OS-native secret storage and a clean CLI developer experience.

Table of Contents

1. Overview
2. Goals and Non-Goals
3. Personas
4. Core Concepts
5. Functional Requirements
6. CLI Design
7. Architecture
8. Platform Keychain Integration
9. Env File Parser
10. Export and Import Format
11. History Tracking
12. Runtime Injection
13. Error Handling
14. Testing Strategy
15. Go Best Practices
16. Phased Implementation Plan

---

1. Overview

`deadenv` is a cross-platform CLI tool written in Go that replaces `.env` files as the mechanism for managing environment variables in development and CI workflows. Instead of storing credentials as plaintext on disk, `deadenv` stores them in the operating system's native secret store — Keychain on macOS, libsecret/GNOME Keyring or KWallet on Linux, Credential Manager on Windows.

Secrets are retrieved at runtime and injected directly into a subprocess or the current shell session. They never touch the filesystem in plaintext.

---

2. Goals and Non-Goals

Goals

- Eliminate plaintext `.env` files from developer machines and repositories.
- Use OS-native authentication (Touch ID, biometrics, device password) as the access gate.
- Provide a minimal, intuitive CLI with a short learning curve for developers already familiar with `.env` conventions.
- Support secure export and import of profiles so credentials can be shared between teammates without transmitting plaintext.
- Maintain a git-backed audit log of structural changes (which keys were added, removed, or modified) without ever persisting raw values.
- Support all common `.env` syntax conventions in the parser.

Non-Goals

- `deadenv` is not a secrets manager for production infrastructure. It is a developer-machine tool. For production, use Vault, AWS Secrets Manager, GCP Secret Manager, etc.
- `deadenv` does not manage secrets across a team in real time. Sharing is a one-time export/import operation.
- `deadenv` does not encrypt the git history repository itself. The history repo stores only key names and metadata — no values.
- `deadenv` does not support secret rotation, expiry, or access policies.
- `deadenv` does not integrate with `.gitignore` automation or IDE plugins in v1.

---

3. Personas

Solo Developer — wants to stop having `.env.local` files that could be accidentally committed. Uses `deadenv run` as a drop-in replacement for `dotenv -e .env`. Primary interaction: `deadenv run myapp -- npm run dev`.

Team Lead — sets up profiles locally and exports them for teammates using the `deadenv export` command. Teammates import with `deadenv import`. Does not want to paste secrets into Slack or email.

New Team Member — receives a `.deadenv` export file and a password via a secure channel. Runs `deadenv import` once and never touches the credentials again.

4. Core Concepts

Profile

A named collection of key-value pairs stored in the OS keychain. A profile is the primary unit of organisation in `deadenv`. It replaces a single `.env` file.

Examples: `myapp-dev`, `myapp-staging`, `payments-service`.

Naming rules: lowercase letters, digits, and hyphens only. No spaces. Max 64 characters.

Key

An environment variable name within a profile. Must follow POSIX conventions: uppercase letters, digits, and underscores; must not start with a digit.

Value

The secret string associated with a key. Never written to disk in plaintext by `deadenv`. Stored exclusively in the OS keychain.

Export File (`.deadenv`)

A portable, encrypted file containing all key-value pairs of a profile. Encrypted with AES-256-GCM using a key derived from a user-chosen sharing password via Argon2id. Intended for one-time sharing between trusted parties.

History Repository

A git repository at `~/.config/deadenv/history/`. Each profile has a corresponding JSON file tracking key names, operations, and timestamps — never values. Changes are auto-committed on every mutation.

---

5. Functional Requirements

FR-01 Profile Management

ID	Requirement
FR-01-1	A user can create a new profile by name.
FR-01-2	A user can list all profiles.
FR-01-3	A user can show the keys in a profile with masked values.
FR-01-4	A user can delete a profile and all its keys from the keychain.

ID	Requirement
FR-01-5	A user can rename a profile.
FR-01-6	A user can copy a profile to a new name.

FR-02 Key Management

ID	Requirement
FR-02-1	A user can set a key in a profile. If the value is omitted from args, `deadenv` prompts with hidden input.
FR-02-2	A user can retrieve a key value, masked by default, with a `--reveal` flag to show plaintext.
FR-02-3	A user can remove a key from a profile.
FR-02-4	All read operations trigger OS-native authentication.
FR-02-5	The OS auth prompt displays a human-readable context string identifying the profile being accessed.

FR-03 Population at Creation

ID	Requirement
FR-03-1	A user can create a profile by providing a path to an existing `.env`-format file.
FR-03-2	A user can create a profile by opening `$VISUAL` or `$EDITOR` (falling back to `vi`) to paste content interactively.
FR-03-3	The editor opens pre-filled with a comment block documenting accepted formats.
FR-03-4	If the editor is saved empty (only comments/whitespace), creation is cancelled cleanly.
FR-03-5	After parsing, the user is shown a summary of parsed keys and prompted to confirm before writing to the keychain.

FR-04 Env File Parsing

ID	Requirement
FR-04-1	Parser accepts `KEY=VALUE`, `KEY = VALUE`, `KEY VALUE`, and `export KEY=VALUE` formats.
FR-04-2	`=` is always the preferred delimiter when present anywhere in the line; space/tab is used only when no `=` exists.
FR-04-3	Values containing `=` (e.g. base64 tokens, API keys) are preserved correctly.
FR-04-4	Double-quoted and single-quoted values strip surrounding quotes.
FR-04-5	Escape sequences (`\"`, `\\`, `\n`, `\t`) are processed inside double-quoted values only.
FR-04-6	Unquoted values strip trailing inline comments (`# comment`). Quoted values do not.
FR-04-7	URLs containing `#` (e.g. `http://example.com#anchor`) are not truncated.
FR-04-8	Lines starting with `#` and blank lines are skipped.
FR-04-9	`KEY=` and `KEY=""` produce an empty string value (valid).
FR-04-10	Unmatched quotes fall back to treating the value as a literal string rather than returning an error.
FR-04-11	Invalid key names (failing POSIX rules) return a descriptive parse error with line number.

FR-05 Runtime Injection

ID	Requirement
FR-05-1	`deadenv run <profile> -- <command>` injects profile variables into a subprocess. Variables do not leak to the parent shell.

ID	Requirement
FR-05-2	`deadenv export <profile>` prints `export KEY=VALUE` shell statements to stdout for use with `eval`.
FR-05-3	`deadenv export` supports `--format` flag with values `shell` (default), `fish`, and `json`.
FR-05-4	`deadenv init --shell=<zsh

FR-06 Export and Import

ID	Requirement
FR-06-1	`deadenv export <profile> --out=<file>` encrypts the profile to a `.deadenv` file.
FR-06-2	The export prompts for a sharing password (separate from OS auth) with confirmation.
FR-06-3	The export file is self-describing: it contains all KDF parameters needed for decryption.
FR-06-4	`deadenv import <file>` decrypts the file and stores the profile in the local keychain.
FR-06-5	On import, the user can override the profile name.
FR-06-6	A wrong password on import produces a clear error, not a panic or a corrupt write.
FR-06-7	The export file format is versioned to allow future migration.

FR-07 History

ID	Requirement
FR-07-1	Every profile mutation (set, unset, delete) is auto-committed to the history git repo.
FR-07-2	Commit messages follow the format: `[<profile>] <op> <KEY>` (e.g. `[myapp] set DATABASE_URL`).

ID	Requirement
FR-07-3	The history JSON file for a profile tracks: key name, operation (`set`/`unset`), timestamp, and a salted SHA-256 hash of the value.
FR-07-4	`deadenv history <profile>` displays a formatted log of changes.
FR-07-5	`deadenv history <profile> --key=<KEY>` filters the log to a single key.
FR-07-6	If git is not installed, history is silently skipped with a one-time warning at tool startup.

6. CLI Design

Built with `github.com/urfave/cli/v2`.

```
deadenv
├── profile
│   ├── new <name> [--from=<file>]
│   ├── list  (alias: ls)
│   ├── show <name> [--reveal]
│   ├── delete <name>  (alias: rm)
│   ├── rename <old> <new>
│   └── copy <src> <dst>
├── set <profile> <KEY> [VALUE]
├── get <profile> <KEY> [--reveal]
├── unset <profile> <KEY>
├── run <profile> -- <command> [args...]
├── export <profile> [--out=<file>] [--format=shell|fish|json]
├── import <file> [--as=<profile>]
├── history <profile> [--key=<KEY>]
└── init [--shell=zsh|bash|fish]
```

Global flags

Flag	Description
`--config <dir>`	Override config directory (default: `~/.config/deadenv`)
`--no-history`	Skip git history commit for this operation

Flag	Description
`--quiet`	Suppress informational output

UX principles

- Every destructive operation (`delete`, `unset`) requires explicit confirmation unless `--yes` is passed.
- Every read that touches the keychain triggers OS auth. There is no session caching in v1.
- Error messages always suggest the corrective command. Example: `Profile "myapp" not found. Create it with: deadenv profile new myapp`.
- The `--` separator in `run` is mandatory and clearly documented to prevent flag parsing conflicts.

---

7. Architecture

Directory layout

```
deadenv/
├── main.go
├── cmd/                    # urfave/cli command definitions (thin layer)
│   ├── profile.go
│   ├── set.go
│   ├── run.go
│   ├── export.go
│   ├── import.go
│   ├── history.go
│   └── init.go
├── internal/
│   ├── keychain/           # OS keychain abstraction
│   │   ├── keychain.go     # interface + shared types
│   │   ├── keychain_darwin.go
│   │   ├── keychain_linux.go
│   │   └── keychain_windows.go
│   ├── parser/             # .env format parser
│   │   ├── parser.go
│   │   └── parser_test.go
│   ├── crypto/             # export encryption/decryption
│   │   ├── crypto.go
│   │   └── crypto_test.go
```

```
│   ├── history/            # git-backed audit log
│   │   ├── history.go
│   │   └── history_test.go
│   ├── profile/            # profile-level operations
│   │   ├── profile.go
│   │   └── profile_test.go
│   └── tui/                # prompts, masked input, confirmations
│       └── tui.go
├── go.mod
├── go.sum
└── Makefile
```

Package responsibilities

`cmd/` — command wiring only. Each file registers subcommands and delegates immediately to `internal/` packages. No business logic.

`internal/keychain` — the platform interface. Defines a `Store` interface. Platform files implement it behind build tags. The rest of the codebase depends only on the interface, never on platform implementations directly.

`internal/parser` — pure function, no I/O, no dependencies. Takes a string, returns `[]EnvPair` or an error. Fully unit-testable.

`internal/crypto` — pure crypto operations. No keychain, no filesystem beyond reading/writing the export file. Fully unit-testable.

`internal/history` — wraps git operations. Designed so the git dependency can be stubbed in tests.

`internal/profile` — orchestrates keychain + history + parser for profile-level operations (create, delete, copy, rename).

`internal/tui` — terminal I/O helpers: hidden password prompt, confirmation prompt, masked value display, summary table.

Interfaces

```go
```

```go
// internal/keychain/keychain.go

package keychain

type Store interface {
    Write(service, account, value string) error
    Read(service, account, prompt string) (string, error)
    Delete(service, account string) error
    List(service string) ([]string, error) // list all accounts under a service
}

// Platform constructor — each platform file implements this
func New() Store
```

```go
// internal/history/history.go

package history

type Recorder interface {
    Record(profile, op, key, valueHash string) error
    Log(profile, key string) ([]Entry, error) // key="" means all keys
}

type Entry struct {
    Profile   string
    Op        string // "set" | "unset" | "delete-profile"
    Key       string
    ValueHash string
    Timestamp time.Time
}
```

---

8. Platform Keychain Integration

macOS

- Use `Security.framework` via cgo.
- Store items under service name `deadenv/<profile>`, account = key name.
- Apply `kSecAccessControlUserPresence` access control at write time. This enforces Touch ID or device password on every read automatically.

- Set `kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly` — items are not exportable to iCloud and cannot be migrated to another device without re-import.
- `kSecUseOperationPrompt` is set to `deadenv wants to access profile "<name>"` so the Touch ID dialog is informative.

Linux

- Target `libsecret` via D-Bus for GNOME Keyring.
- Fall back to `kwallet-query` (KWallet) if `libsecret` is unavailable.
- Attribute schema: `{ "application": "deadenv", "profile": "<name>", "key": "<KEY>" }`.
- If neither is available, print a clear error and exit. Do not fall back to filesystem storage.

Windows

- Use `wincred` (Windows Credential Manager) via the `github.com/danieljoos/wincred` package.
- Target name: `deadenv/<profile>/<KEY>`.
- Windows Hello prompts naturally when the system is configured for it.

Build tags

Each platform file uses `//go:build <platform>` at the top. The `keychain.New()` constructor is the only entry point the rest of the codebase calls.

---

9. Env File Parser

The parser is a standalone pure function in `internal/parser`.

Parsing rules (in order)

1. Split input on newlines.
2. Trim leading/trailing whitespace from each line.
3. Skip blank lines and lines where the first non-whitespace character is `#`.
4. Strip `export ` prefix (case-sensitive) if present, then re-trim.
5. Delimiter detection:
    - Search for the first `=` in the line.
    - If `=` is found: key = trimmed text before it, value = everything after it processed by `parseValue`.

- If no `=` is found: search for the first space or tab. If found, split there. If neither found, key = whole line, value = `""`.

6. Validate key with `isValidKey`. Return error with line number on failure.

`parseValue` rules

1. Trim leading whitespace.
2. If first character is `"` or `'`: find the last matching quote character (not the first — handles `"abc=="` correctly). Extract content between the outermost quotes. For double-quoted values, process escape sequences: `\"`, `\\`, `\n`, `\t`. Return inner string.
3. If unmatched quote: return the entire string as-is (lenient mode).
4. If unquoted: search for `  #` (space + hash). If found, truncate there. Return trimmed remainder.

Key validation

```
^[A-Za-z_][A-Za-z0-9_]*$
```

Canonical test table

Input line	Key	Value	Notes
`KEY=VALUE`	`KEY`	`VALUE`	basic
`KEY = VALUE`	`KEY`	`VALUE`	spaces around `=`
`KEY VALUE`	`KEY`	`VALUE`	space delimiter
`export KEY=VALUE`	`KEY`	`VALUE`	export prefix
`API_KEY=abc123==`	`API_KEY`	`abc123==`	`=` in value
`API_KEY = abc123==`	`API_KEY`	`abc123==`	spaces + `=` in value
`KEY="quoted==value"`	`KEY`	`quoted==value`	double-quoted
`KEY='quoted==value'`	`KEY`	`quoted==value`	single-quoted
`KEY=value # comment`	`KEY`	`value`	inline comment stripped

Input line	Key	Value	Notes
`KEY="value # not a comment"`	`KEY`	`value # not a comment`	quoted, hash preserved
`KEY=http://host#anchor`	`KEY`	`http://host#anchor`	URL hash preserved
`KEY=`	`KEY`	``	empty value
`KEY=""`	`KEY`	``	explicit empty
`KEY="has \"escaped\""`	`KEY`	`has "escaped"`	escape sequences
`# comment`	—	—	skipped
	—	—	blank, skipped

10. Export and Import Format

File extension

`.deadenv`

Format

JSON, human-readable, version-tagged.

```json
{
  "version": 1,
  "profile": "myapp",
  "created_at": "2025-04-19T10:00:00Z",
  "kdf": {
    "algorithm": "argon2id",
    "salt": "<base64>",
    "time": 2,
    "memory": 131072,
    "threads": 4
  },
  "nonce": "<base64>",
  "ciphertext": "<base64>"
}
```

Encryption scheme

- **KDF:** Argon2id, time=2, memory=128MB, threads=4, key length=32 bytes.
- **Cipher:** AES-256-GCM. The GCM authentication tag detects both wrong passwords and file tampering with the same error path — do not distinguish between them in the error message (avoids oracle attacks).
- **Nonce:** 12 bytes random per export.
- **Salt:** 32 bytes random per export.
- **Plaintext:** `json.Marshal([]EnvPair)`.
- **File permissions:** written as `0600`.

Import behaviour

- Read and decode the JSON envelope.
- Re-derive key using KDF params from the file.
- Decrypt and verify GCM tag. On failure: `decryption failed — wrong password or file is corrupted`.
- Parse the plaintext `[]EnvPair`.
- Prompt user to confirm or override the profile name.
- Show summary of keys to be imported.
- Write to keychain.
- Record in history.

---

11. History Tracking

Repository location

`~/.config/deadenv/history/` — initialised as a git repo on first use.

Per-profile file

`~/.config/deadenv/history/<profile>.json`

```
json
```

```
{
  "profile": "myapp",
  "keys": {
    "DATABASE_URL": {
      "op": "set",
      "value_hash": "<hex>",
      "updated_at": "2025-04-19T10:00:00Z"
    },
    "OLD_KEY": {
      "op": "unset",
      "value_hash": "",
      "updated_at": "2025-04-18T08:00:00Z"
    }
  }
}
```

Value hash

`SHA-256(salt + value)` where salt is a fixed per-installation random string stored at `~/.config/deadenv/history/.salt`. The hash lets users confirm whether a value changed between two commits without revealing the value itself.

Commit message format

```
[myapp] set DATABASE_URL
[myapp] unset OLD_KEY
[myapp] delete profile
[myapp] import (8 keys)
```

`deadenv history` output format

```
Profile: myapp
──────────────────────────────────────────────────

2025-04-19 10:00  set     DATABASE_URL   (hash: a3f1...)
2025-04-18 09:00  set     API_KEY        (hash: 7c2d...)
2025-04-17 14:00  unset   OLD_KEY
```

Git not available

If `git` is not found on `PATH`, history is silently skipped and a one-time notice is printed: `git not found — history tracking disabled. Install git to enable it.`

`deadenv run`

```bash
deadenv run <profile> -- <command> [args...]
```

Implementation:

1. Authenticate and read all key-value pairs for the profile from the keychain.
2. Copy `os.Environ()` into a new slice.
3. Append or overwrite with profile pairs.
4. `exec.Command(command, args...)` with the new env slice.
5. Wire `Stdin`, `Stdout`, `Stderr` to the parent process.
6. Use `cmd.Run()`. Propagate the exit code using `os.Exit` with the process exit code.

Profile variables that collide with existing env vars **overwrite** them, consistent with `.env` tool conventions.

`deadenv export` (for eval)

```bash
eval $(deadenv export myapp)                # bash/zsh
deadenv export myapp --format=fish | source # fish
deadenv export myapp --format=json          # machine-readable
```

Values are shell-escaped before output. Use `shlex`-style quoting: wrap in single quotes, escape embedded single quotes as `'\''`.

Shell hook (`deadenv init`)

```bash
deadenv init --shell=zsh
```

Prints to stdout:

```zsh
```

```
# deadenv shell hook — paste into ~/.zshrc
deadenv() {
  if [[ "$1" == "use" ]]; then
    eval $(command deadenv export "$2")
  else
    command deadenv "$@"
  fi
}
```

The user is responsible for adding it to their rc file. `deadenv init` does not modify files automatically.

---

13. Error Handling

Principles

- All errors returned from `internal/` packages are wrapped with `fmt.Errorf("context: %w", err)` to preserve the chain.
- `cmd/` layer is the only place errors are printed to the user. Never `log.Fatal` inside `internal/`.
- User-facing errors are written to `stderr`. Normal output goes to `stdout`.
- Sensitive values must never appear in error messages.

Exit codes

Code	Meaning
0	Success
1	General error (bad args, not found, etc.)
2	Authentication failure (OS keychain denied)
3	Decryption failure (wrong password or corrupt file)
4	Parse error (malformed env content)
127	Command not found (propagated from `run`)
N	Exit code propagated from the subprocess in `run`

Sentinel errors

Define typed sentinel errors in each package for conditions callers need to branch on:

```go
var (
    ErrProfileNotFound = errors.New("profile not found")
    ErrKeyNotFound     = errors.New("key not found")
    ErrAuthDenied      = errors.New("authentication denied")
    ErrDecryptFailed   = errors.New("decryption failed")
    ErrEmptyContent    = errors.New("no variables found")
)
```

---

14. Testing Strategy

Philosophy

- Test behaviour, not implementation. Tests should not know about struct field names or internal state.
- Pure functions (`parser`, `crypto`) get exhaustive table-driven unit tests.
- I/O-dependent packages (`keychain`, `history`) use interfaces so behaviour can be tested with fakes.
- Integration tests run against real OS keychain and real git, gated behind a build tag `//go:build integration`.

Unit tests

`internal/parser`

Table-driven tests covering every row in the canonical test table, plus edge cases:

- Multi-line input with a mix of valid and comment lines.
- Input with Windows-style `\r\n` line endings.
- Consecutive `=` signs in values.
- Values that are only whitespace.
- Keys that start with a digit (should error).
- `export` prefix with extra whitespace.

```go
```

```go
func TestParseEnvContent(t *testing.T) {
    tests := []struct {
        name    string
        input   string
        want    []EnvPair
        wantErr bool
    }{
        // ... table rows
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := ParseEnvContent(tt.input)
            if (err != nil) != tt.wantErr {
                t.Fatalf("ParseEnvContent() error = %v, wantErr %v", err, tt.wa
            }
            if !tt.wantErr && !reflect.DeepEqual(got, tt.want) {
                t.Errorf("got %v, want %v", got, tt.want)
            }
        })
    }
}
```

`internal/crypto`

- Round-trip: `Export` → `Import` produces identical pairs.
- Wrong password returns `ErrDecryptFailed`.
- Tampered ciphertext (flip a byte) returns `ErrDecryptFailed`.
- Tampered nonce returns `ErrDecryptFailed`.
- Each export with the same password produces a different ciphertext (nonce randomness).
- Version mismatch in envelope returns a clear error.
- KDF params in output file match the constants used.

`internal/parser` **fuzz test**

```go
```

```go
func FuzzParseEnvContent(f *testing.F) {
    f.Add("KEY=VALUE\n")
    f.Add("KEY = VALUE\n")
    f.Fuzz(func(t *testing.T, data string) {
        // must never panic, regardless of input
        _, _ = ParseEnvContent(data)
    })
}
```

Run with `go test -fuzz=FuzzParseEnvContent -fuzztime=30s ./internal/parser/`.

Fake implementations

go

```go
// internal/keychain/fake.go (not a build-tagged file, available in all tests)

package keychain

type FakeStore struct {
    data map[string]map[string]string // service → account → value
    Err  error // set to simulate errors
}

func NewFake() *FakeStore {
    return &FakeStore{data: make(map[string]map[string]string)}
}

func (f *FakeStore) Write(service, account, value string) error {
    if f.Err != nil { return f.Err }
    if f.data[service] == nil { f.data[service] = make(map[string]string) }
    f.data[service][account] = value
    return nil
}

func (f *FakeStore) Read(service, account, prompt string) (string, error) {
    if f.Err != nil { return "", f.Err }
    v, ok := f.data[service][account]
    if !ok { return "", ErrKeyNotFound }
    return v, nil
}

func (f *FakeStore) Delete(service, account string) error {
    if f.Err != nil { return f.Err }
    delete(f.data[service], account)
    return nil
}

func (f *FakeStore) List(service string) ([]string, error) {
    if f.Err != nil { return nil, f.Err }
    var keys []string
    for k := range f.data[service] { keys = append(keys, k) }
    return keys, nil
}
```

`internal/profile` tests

Use `FakeStore` and a fake `history.Recorder`:

- Create profile with valid pairs → keys stored under correct service name.

- Create profile with empty pairs → returns `ErrEmptyContent`, nothing written.
- Delete profile → all keys removed from keychain, history entry recorded.
- Copy profile → all keys duplicated under new service name.
- Rename = copy + delete.

`internal/history` tests

- Use a temp directory for the git repo.
- Record a set operation → JSON file updated, git log has one commit.
- Record a second set → JSON updated, git log has two commits.
- Record an unset → op field is `"unset"`, value_hash is empty.
- Log with key filter → only returns entries for that key.
- `git` not on PATH → returns `nil` error, silently no-ops.

Integration tests

Tag: `//go:build integration`

Run with: `go test -tags=integration ./...`

- macOS only: write a key to Keychain with `FakeStore` replaced by the real implementation, read it back.
- Run `deadenv run` against a real subprocess and assert env vars are present in its output.
- Full export → import round-trip with real files and real keychain.

These tests require a real machine, Touch ID interaction, and are excluded from CI by default. They exist for manual pre-release verification.

Coverage target

- `internal/parser`: 100% line coverage.
- `internal/crypto`: 100% line coverage.
- `internal/profile` (with fakes): ⩾ 90% line coverage.
- `internal/history` (with temp git repo): ⩾ 85% line coverage.
- `cmd/`: not measured — these are thin wiring layers. Covered by integration tests.

Test helpers

```go
```

```go
// testutil/testutil.go
package testutil

import "testing"

func Must[T any](t *testing.T, v T, err error) T {
    t.Helper()
    if err != nil { t.Fatalf("unexpected error: %v", err) }
    return v
}

func MustNoErr(t *testing.T, err error) {
    t.Helper()
    if err != nil { t.Fatalf("unexpected error: %v", err) }
}
```

---

15. Go Best Practices

Module and dependency management

- Module path: `github.com/<username>/deadenv`
- Keep `go.mod` tidy. Run `go mod tidy` before every commit.
- Minimise dependencies. Direct dependencies only for: `github.com/urfave/cli/v2`, `golang.org/x/crypto` (Argon2id), `golang.org/x/term` (hidden input).
- Platform-specific keychain bindings via cgo (darwin) or exec (linux) — no third-party keychain libraries in v1.

Code style

- `gofmt` enforced. No exceptions.
- `golangci-lint` with at minimum: `errcheck`, `staticcheck`, `govet`, `unused`, `gosec`.
- All exported symbols have doc comments.
- No `init()` functions.
- No global mutable state. Dependencies are passed explicitly (constructor injection).

Error handling

- Every error from a called function is handled. Never `_` an error return.

- Use `%w` for wrapping: `fmt.Errorf("reading profile: %w", err)`.
- Use `errors.Is` and `errors.As` for matching — never string comparison on error messages.
- Errors that indicate programmer mistakes (nil pointer, wrong argument type) use `panic`. All other errors propagate.

Concurrency

- v1 has no concurrent operations — the CLI is inherently sequential.
- Do not add goroutines speculatively. Introduce them only when a concrete use case demands it.

Security

- Secrets in memory: use `[]byte` rather than `string` where possible, and zero the slice after use with `for i := range b { b[i] = 0 }`. Note: Go's GC does not guarantee immediate collection, but zeroing reduces the window.
- Never log a secret value. Ensure `%v`/`%s` formatting of types containing values cannot reach logs.
- Temp files for editor sessions: create with `os.CreateTemp`, defer `os.Remove` immediately after creation.
- Export files: write with `os.WriteFile(path, data, 0600)` — never `0644`.
- Shell export: always quote values before printing. Use single-quote wrapping with `'\''` escaping for embedded single quotes.

Makefile targets

```makefile
```

```makefile
.PHONY: build test lint fuzz integration clean

build:
    go build -o bin/deadenv ./...

test:
    go test -race ./...

lint:
    golangci-lint run ./...

fuzz:
    go test -fuzz=FuzzParseEnvContent -fuzztime=60s ./internal/parser/

integration:
    go test -tags=integration -v ./...

clean:
    rm -rf bin/
```

---

16. Phased Implementation Plan

Each phase produces a working, testable increment. No phase depends on unfinished work from a later phase.

Phase 1 — Project Skeleton and Parser

Goal: Compilable project with a fully tested parser. Nothing talks to a keychain yet.

Deliverables:

- `go.mod` with `urfave/cli/v2` and `golang.org/x/crypto` declared.
- `main.go` with a root `cli.App` that prints help.
- `internal/parser/parser.go` — `ParseEnvContent`, `parseValue`, `isValidKey`.
- `internal/parser/parser_test.go` — full table-driven tests and fuzz seed corpus.
- `testutil/testutil.go` — `Must`, `MustNoErr` helpers.
- `Makefile` with `build`, `test`, `lint`, `fuzz` targets.

Independently testable: `go test ./internal/parser/` passes 100%.

Done when: `go test -race ./...` is green. `go test -fuzz=FuzzParseEnvContent -fuzztime=30s ./internal/parser/` finds no panics.

Phase 2 — Keychain Interface and Fake

Goal: Define the storage contract and a fake that all subsequent phases can test against.

Deliverables:

- `internal/keychain/keychain.go` — `Store` interface, sentinel errors, `EnvPair` type.
- `internal/keychain/fake.go` — `FakeStore` implementation.
- `internal/keychain/keychain_darwin.go` — stub that returns `errors.New("not implemented")` (build tag `darwin`). Real cgo implementation deferred to Phase 5.
- `internal/keychain/keychain_linux.go` — same stub (build tag `linux`).
- `internal/keychain/keychain_windows.go` — same stub (build tag `windows`).

Independently testable: `FakeStore` satisfies the `Store` interface at compile time. Verified with `var _ keychain.Store = &keychain.FakeStore{}`.

Phase 3 — Profile Operations (with Fake Keychain)

Goal: Full profile and key management logic, tested entirely against `FakeStore`.

Deliverables:

- `internal/profile/profile.go` — `Create`, `Delete`, `Copy`, `Rename`, `ListKeys`, `SetKey`, `GetKey`, `UnsetKey`. Accepts `keychain.Store` and `history.Recorder` interfaces.
- `internal/profile/profile_test.go` — all operations tested with `FakeStore` and a fake recorder.
- `internal/tui/tui.go` — `PromptHidden(label string) (string, error)`, `PromptConfirm(msg string) bool`, `PrintSummary(pairs []EnvPair)`, `MaskValue(v string) string`.

Independently testable: `go test ./internal/profile/` — all ops green with fake dependencies.

Phase 4 — History Tracking

Goal: git-backed audit log, independently testable with temp directories.

- `internal/history/history.go` — `GitRecorder` implementing `Recorder`. Uses `os/exec` to shell out to git.
- `internal/history/noop.go` — `NoopRecorder` for when git is unavailable.
- `internal/history/history_test.go` — tests using `t.TempDir()` as the repo root, verifying JSON output and git log.

Independently testable: `go test ./internal/history/` — runs real git in temp dirs, no keychain.

Phase 5 — Real Keychain Implementations

Goal: Actual OS secret storage behind the `Store` interface.

Deliverables:

- `internal/keychain/keychain_darwin.go` — full cgo implementation with `kSecAccessControlUserPresence`.
- `internal/keychain/keychain_linux.go` — `secret-tool` exec implementation with GNOME Keyring.
- `internal/keychain/keychain_windows.go` — `wincred` implementation.
- Build matrix CI confirming each platform file compiles (even if integration tests are not run).

Independently testable: `go build ./...` succeeds on each platform. Integration tests (`-tags=integration`) verify real reads/writes on developer machines.

Phase 6 — Editor and File Import

Goal: `deadenv profile new` with both `--from` file path and `$EDITOR` flow.

Deliverables:

- `internal/profile/populate.go` — `FromFile(path string) ([]EnvPair, error)` and `FromEditor(template string) ([]EnvPair, error)`.
- Editor template string with comment block documenting all accepted formats.
- `FromEditor` opens `$VISUAL` or `$EDITOR` or `vi`, writes temp file, defers removal, reads back after exit.
- `cmd/profile.go` — `profile new` command wired up.

Independently testable: `FromFile` is fully unit-testable. `FromEditor` can be tested by setting `EDITOR=cat` in the test environment so no interactive terminal is needed.

Phase 7 — Crypto (Export / Import)

Goal: Encrypted `.deadenv` file export and import.

Deliverables:

- `internal/crypto/crypto.go` — `Export(profile string, pairs []EnvPair, password, outPath string) error` and `Import(path, password string) ([]EnvPair, string, error)`.
- `internal/crypto/crypto_test.go` — round-trip, wrong password, tampered file, nonce uniqueness tests.
- `cmd/export.go` — wires `deadenv export` including OS auth prompt (read from keychain) + sharing password prompt.
- `cmd/import.go` — wires `deadenv import` including profile name override and keychain write.

Independently testable: `go test ./internal/crypto/` — no keychain, no filesystem beyond temp files.

Phase 8 — Runtime Injection

Goal: `deadenv run` and `deadenv export` for shell eval.

Deliverables:

- `cmd/run.go` — reads profile from keychain, builds child env, execs command, propagates exit code.
- `cmd/export.go` (extend) — add `--format=shell|fish|json` output modes with proper shell escaping.
- `cmd/init.go` — prints shell hook snippet for `zsh`, `bash`, `fish`.
- Tests for shell escaping edge cases: values with spaces, single quotes, newlines, equals signs.

Independently testable: `deadenv run` tested by running `/usr/bin/env` as the subprocess and asserting its output contains the expected vars. No real keychain needed if `FakeStore` is injected.

Phase 9 — Polish and CLI Wiring

Goal: All commands wired, error messages hardened, UX complete.

Deliverables:

- All `cmd/` files complete with proper flag definitions, help text, and usage examples.
- Error messages for every sentinel error type mapped to a human-readable string with a suggested corrective command.
- `--quiet`, `--no-history`, `--yes` global flags implemented.
- `deadenv profile list` and `deadenv profile show` with masked output table.
- `deadenv history` with formatted log and `--key` filter.
- README with installation, quickstart, and command reference.

Independently testable: End-to-end smoke test script (`scripts/smoke_test.sh`) that creates a profile, sets keys, runs a subprocess, exports, deletes, and imports — using `EDITOR=cat` and a real keychain on the dev machine.

Phase 10 — Release

Goal: Distributable binaries.

Deliverables:

- `goreleaser` configuration targeting macOS (arm64, amd64), Linux (amd64), Windows (amd64).
- GitHub Actions CI: `test` and `lint` on every PR. `release` on tag push.
- `go install github.com/<username>/deadenv@latest` works.
- Homebrew tap formula (optional but recommended for macOS adoption).

Appendix A — Dependency Summary

Package	Purpose	Justification
`github.com/urfave/cli/v2`	CLI framework	Established, minimal, good subcommand support
`golang.org/x/crypto`	Argon2id, bcrypt	Official extended crypto — needed for KDF

Package	Purpose	Justification
`golang.org/x/term`	Hidden terminal input	Official extended lib — cross-platform password prompt

All other requirements are met by the Go standard library.

Appendix B — File Format Version History

Version	Changes
1	Initial format. Argon2id KDF, AES-256-GCM cipher.

Future format changes increment the `version` field. The `Import` function checks this field first and returns a clear error for unsupported versions, enabling migration tooling to be added later.